

Part 1:

1. Testbench architecture and strategy:

- Draw a block diagram of a testbench architecture.

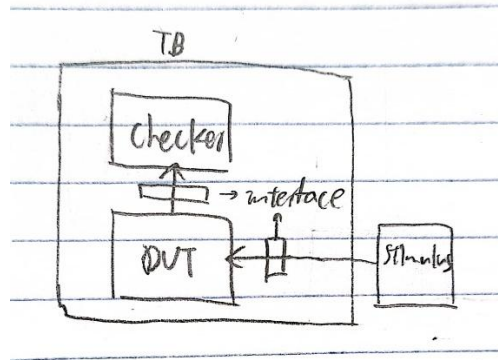


Figure 1: DPI

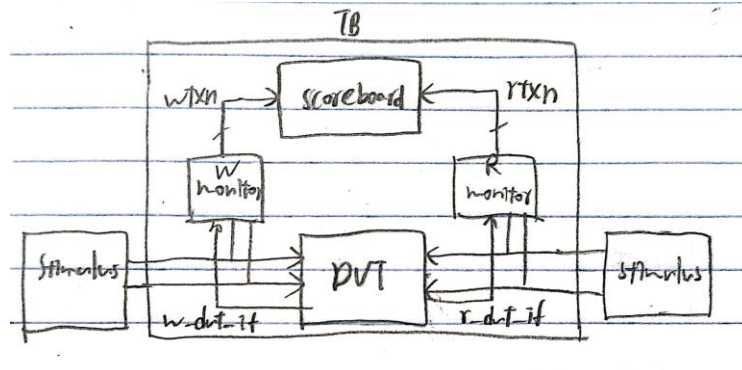


Figure 2: SV

- high-level methodology (simulation, formal) and sign-off metrics strategy: The fifo is simulated in two scenarios targeting 80% line coverage and 50% toggle coverage. There are two verification structures. The DPI architecture connects the DUT with the checker module through interface, as shown in figure 1. The SV architecture connects the driver with two monitors (figure 2), one for the write side of the FIFO, the other for the read side using their respective interfaces. These monitors send transactions through the read and write mailboxes to the scoreboard, where an ideal queue is used as the golden model to compare with the transaction contents.

2. Stimulus and coverage plan:

Feature:

The ratio between read and write clock is 1.25. The rising edges of rclk and wclk will be synchronized, and the fifo finishes reset after the first negative edge of

the read clock. The read task asserts rinc at the negative edge of rclk until a specified number of rclk has passed, and rinc return to 1'b0 at a negative edge of rclk. The write task asserts winc and prepares wdata at the negative edge of wclk until a number of wclks have passed, and winc return to 1'b0 at the negative wclk edge.

1. Basic:. For the basic scenario, read and write are run simultaneously for 40 rclk and wclk respectively. The rempty signal compares the read pointer and the synchronized write pointer, which is a delayed version of the actual write pointer, so the rempty signal will be asserted initially, and the read pointer should remain constant until the synchronized write pointer arrives. The fifo wfull signal is asserted once the difference between synchronized read pointer and write pointer reaches the fifo size. Hence, the burst size is 40, and within 40 wclk cycles, $40 \times \frac{4}{5} = 32$ data are read. Therefore, the FIFO should not be full.
2. Bug: Turning to the bug scenario, the bug is injected by disabling the write operation in the fifo memory for every 4 data. This is achieved by asserting the bug enable signal every time the next write pointer reaches a multiple of 4. The simulation performs write for 16 data and read 16 data. Hence the winc is asserted for 16 wclk cycles and interrupted 4 times, and at the first rclk negative edge after the write operation, read is performed.

3. Checking plan:

- a. Basic scenario:
 1. Wfull is not asserted.
 2. Rempty is asserted after the first three read operations.
 3. Rdata matches the golden model.
- b. Bug scenario:
 1. Wfull should not be asserted.
 2. Rempty turn 0 after the first three rclk cycles.
 3. Rdata mismatches the rdata from the golden model by 1 data after the first period, and 2 data after the second period...,etc.

Part 2:

1. Briefly describe your C/C++ golden model and its functions. Highlight relevant code snippets in your report.

- a. The C golden model implements a queue with a struct including a integer array, read and write pointer and size.

```
typedef struct{
    int q[FIFO_SIZE];
    int read;
    int write;
    int size;
} Queue;
```

b. The methods include

a. front(): return the first element in the queue.

```
int front(){
    return queue.q[queue.read];
}
```

b. Pop(): pop the first element, decrease the size, and return this element.

```
int pop(){
    if(queue.size==0){
        return queue.q[queue.read];
    }else{
        queue.size=queue.size-1;
        int val = queue.q[queue.read];
        queue.read=(queue.read+1)%FIFO_SIZE;
        return val;
    }
}
```

c. Push(): attach the new element to the end of the queue and increase the size.

```
void push(int val){
    if(queue.size==FIFO_SIZE){
        return;
    }else{
        queue.size=queue.size+1;
        queue.q[queue.write]=val;
        queue.write=(queue.write+1)%FIFO_SIZE;
    }
}
```

d. Is_empty(): if the size==0.

```
svLogic is_empty(){
    return queue.size==0;
}
```

e. Is_full(): if the size==16.

```
svLogic is_full(){
    return queue.size==FIFO_SIZE;
}
```

f. Rreset(): reset size to 0 and read pointer to 0.

```
void rreset(){
    queue.size=0;
    queue.read=0;
}
```

g. Wreset(): reset size to 0 and write pointer to 0.

```
void wreset(){
    queue.size=0;
    queue.write=0;
}
```

h. Near_empty(): size<=4

```
svLogic near_empty(){  
    return queue.size<=FIFO_SIZE/4;  
}
```

i. Near_full(): size>=12

```
svLogic near_full(){  
    return queue.size>=FIFO_SIZE*3/4;  
}
```

2. Briefly describe the checker module implementation and highlight relevant code snippets in the report.

The checker module is connected to the dut in the testbench, and all signals are input. The DPI imported functions are called to implement the golden model at the positive clock edges, and comparisons are performed at the negative clock edges.

a. The values from the golden model are written at the rclk positive edge.

```
always begin  
    @(posedge rclk);  
    if(rrst_n==1'b0)rreset();  
    else begin  
        if(rinc&rrst_n&(~rempty))pop();  
        rdata_gold=front();  
        almost_empty_gold=near_empty();  
        rempty_gold=is_empty();  
    end  
end
```

b. Comparison is performed at the negative rclk edge when the dut signals have been overwritten.

```
if(rdata!=rdata_gold)$error("wrong data: %d, expected data %d", rdata, rdata_gold);  
else $display("read data %d:", rdata);  
//if(almost_empty!=almost_empty_gold)$error("almost empty flag: %b", almost_empty_gold);  
if(rempty!=rempty_gold)$error("rempty flag: %b", rempty_gold);
```

c. The write data is pushed into the golden model at the positive wclk edge, and bug enable condition is included.

```
always begin  
    @(posedge wclk);  
    if(wrst_n==1'b0)begin  
        wreset();  
    end  
    else begin  
        if(wrst_n&(~wfull)&(winc^bug_en))push(wdata);  
        almost_full_gold=near_full();  
        wfull_gold=is_full();  
    end  
end
```

d. The wfull signal is compared at the negative wclk edge.

```
@(negedge wclk);  
//if(almost_full!=almost_full_gold)$error("almost full flag: %b", almost_full_gold);  
if(wfull!=wfull_gold)$error("wfull flag: %b", wfull_gold);
```

3. Attach screenshots of the terminal output and Verdi waveform window for Task 4(a).

```
read data  x:
read data  x:
read data  1:
Error: "FIFO_TB_2.sv", 56: FIFO_testbench_dpi.dut_check: at time 30
rempty flag: 0
read data  1:
Error: "FIFO_TB_2.sv", 56: FIFO_testbench_dpi.dut_check: at time 40
rempty flag: 0
read data  1:
read data  2:
read data  3:
read data  4:
read data  5:
read data  6:
read data  7:
read data  8:
read data  9:
read data 10:
```

Figure 1: terminal output, the rempty flag mismatches the golden model because of the synchronization of the write pointer.

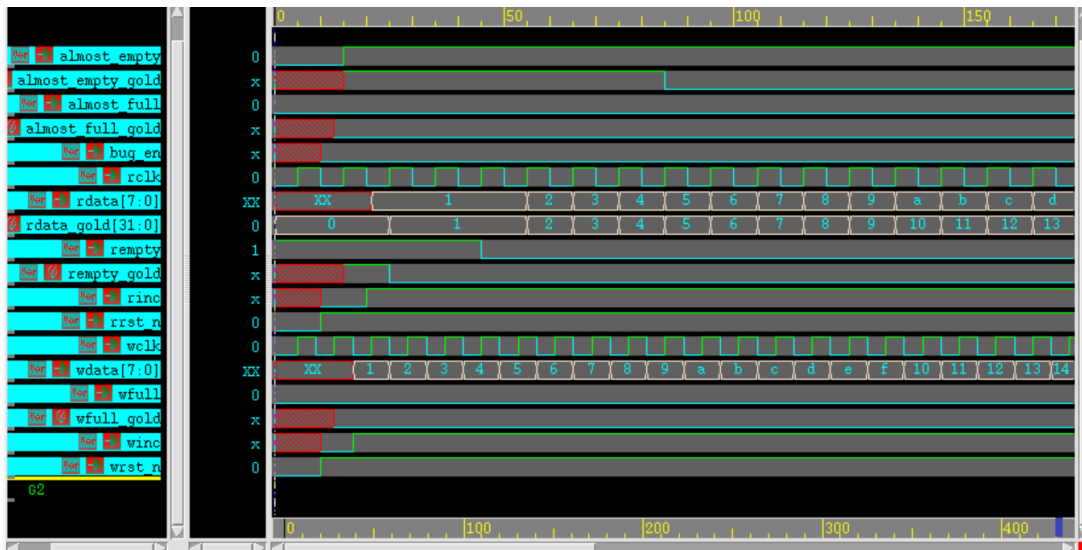


Figure 2: Verdi simulation waveform

4. Attach screenshots of terminal output displaying the error/mismatch and the Verdi simulation waveforms showing a mismatch in the checker for Task 4(b).


```

class rtransaction;
  bit almost_empty, rempty, rinc;
  bit [`DSIZE-1:0] rdata;
  function new();
  endfunction
endclass

class wtransaction;
  bit almost_full, wfull, winc;
  bit [`DSIZE-1:0] wdata;
  bit bug_en;
  function new();
  endfunction
endclass

```

2. Briefly describe the monitor implementation. Highlight relevant code snippets in your report.

The read and write monitors are constructed by the read and write interfaces respectively. At positive edge of the clock, a new transaction is created, and since enable read and enable write is asserted half a clock cycle ago, we can sample this signal at the positive edge. Later, at the negative edge, the fifo output signals are created. Additionally, in bug mode, if winc is 0 and bug enable is asserted, the golden model will continue storing data, as specified in the “wmonitor”.

```

class rmonitor;
  virtual r_dut_if r_dut_vif;
  rtransaction txn;
  mailbox #(rtransaction) txn_mail;
  function new(virtual r_dut_if r_dut_vif);
    this.r_dut_vif = r_dut_vif;
  endfunction
  task run();
    forever begin
      @(posedge r_dut_vif.rclk);
      txn = new();
      txn.rinc=r_dut_vif.rinc&r_dut_vif.rrst_n&(~r_dut_vif.rempty);
      @(negedge r_dut_vif.rclk);
      txn.rdata=r_dut_vif.rdata;
      txn.rempty=r_dut_vif.rempty;
      txn.almost_empty=r_dut_vif.almost_empty;
      txn_mail.put(txn);
    end
  endtask
endclass

```

```

class wmonitor;
  virtual w_dut_if w_dut_vif;
  mailbox #(wtransaction) txn_mail;
  wtransaction txn;
  function new(virtual w_dut_if w_dut_vif);
    this.w_dut_vif = w_dut_vif;
  endfunction
  task run();
    forever begin
      @(posedge w_dut_vif.wclk);
      txn = new();
      txn.winc=(w_dut_vif.winc&w_dut_vif.bug_en)&w_dut_vif.wrst_n&(!w_dut_vif.wfull);
      txn.wdata=w_dut_vif.wdata;
      @(negedge w_dut_vif.wclk);
      txn.wfull=w_dut_vif.wfull;
      txn.almost_full=w_dut_vif.almost_full;
      txn_mail.put(txn);
    end
  endtask
endclass

```

3. Briefly describe the scoreboard implementation and reference model inside it. Highlight relevant code snippets in your report.

The scoreboard is a class containing read and write mail boxes, which receives read and write transactions from their respective monitors. Rtxn and wtxn are the current transactions being executed. The golden model is a queue named “bounded_q”, which can take at most 40 data.

```

class scoreboard;
  mailbox #(rtransaction) rtxn_mail;
  mailbox #(wtransaction) wtxn_mail;
  rtransaction rtxn;
  wtransaction wtxn;
  int rdata_gold;
  int bounded_q[$:`NUM];
  int nth_read;
  int nth_write;

  function print_queue();
    $display("#%d,%d,%d,%d,%d,%d",
      nth_read, bounded_q[0], bounded_q[1], bounded_q[2], bounded_q[3], bounded_q[4]);
  endfunction

```

There are two run tasks for read and write. The read run task examines the read transactions by comparing the read data to the front element in the golden model; while the write run task process write transactions and push in element depending on the write enable signal from the transaction. The two tasks are executed in parallel in the testbench and repeat themselves until the end of the simulation.

```

task rrun();
  nth_read=0;
  forever begin
    rtxn_mail.get(rtxn);
    //print_queue();
    //rdata_gold=bounded_q[0];
    if(rtxn.rinc)begin
      bounded_q.pop_front();
      nth_read=nth_read+1;
    end
    rdata_gold=bounded_q[0];
    if(rdata_gold!=rtxn.rdata)
      $display("wrong read data:%d, expect: %d", rtxn.rdata,rdata_gold);
    else
      $display("%d th read: %d", nth_read, rtxn.rdata);
    if((bounded_q.size()==0)^rtxn.empty)
      $error("empty: %d", rtxn.empty);
    if((bounded_q.size())<=4)^rtxn.almost_empty)
      $error("almost empty: %d", rtxn.almost_empty);
  end
endtask

task wrun();
  nth_write=0;
  forever begin
    wtxn_mail.get(wtxn);
    if(wtxn.winc)begin
      bounded_q.push_back(wtxn.wdata);
      // $display("write in:%d", wtxn.wdata);
      nth_write=nth_write+1;
    end
    if((bounded_q.size()==16)^wtxn.wfull)
      $error("wfull:%d", wtxn.wfull);
    end
    if((bounded_q.size())>=4)^wtxn.almost_full)
      $error("almost full: %d", wtxn.almost_full);
  endtask
endclass

```

4. Attach screenshots of the terminal output and Verdi waveform window for Task 5(a).

```

0 th read: 0
Error: ".RTL/FIFO_TB3.sv", 124: $unit::\scoreboard::rrun : at time 10
almost empty: 0
0 th read: 0
0 th read: 1
Error: ".RTL/FIFO_TB3.sv", 122: $unit::\scoreboard::rrun : at time 30
empty: 1
0 th read: 1
Error: ".RTL/FIFO_TB3.sv", 122: $unit::\scoreboard::rrun : at time 40
empty: 1
0 th read: 1
1 th read: 2
2 th read: 3
3 th read: 4
4 th read: 5
Error: ".RTL/FIFO_TB3.sv", 124: $unit::\scoreboard::rrun : at time 90
almost empty: 1
5 th read: 6
Error: ".RTL/FIFO_TB3.sv", 124: $unit::\scoreboard::rrun : at time 100
almost empty: 1
6 th read: 7
Error: ".RTL/FIFO_TB3.sv", 124: $unit::\scoreboard::rrun : at time 110
almost empty: 1
7 th read: 8

```


Figure 1: terminal output. The empty signal and almost empty signals are based on the synchronized write pointer, which is delayed due to the synchronization block.

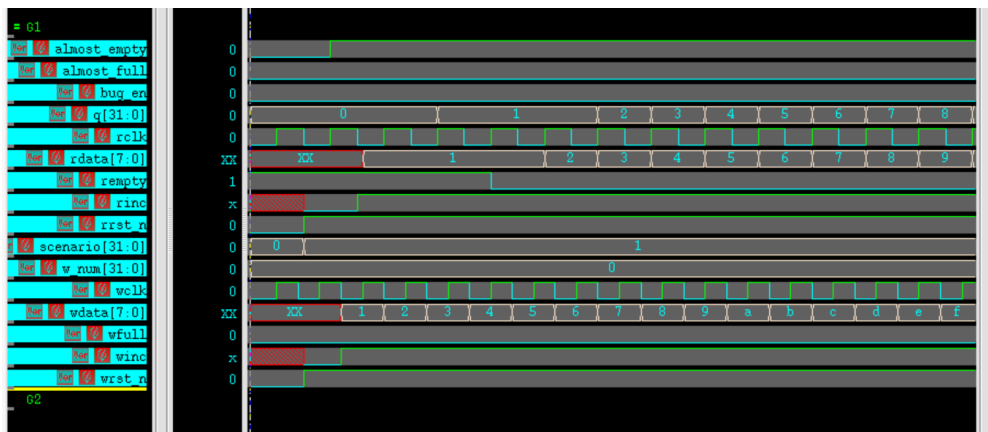


Figure 2: Verdi simulation waveform, where q is the golden model output (captured at posedge of rclk) from the scoreboard, which corresponds to rdata.

5. Attach screenshots of terminal output displaying the error/mismatch and the Verdi simulation waveforms showing a mismatch in the scoreboard for Task 5(b).

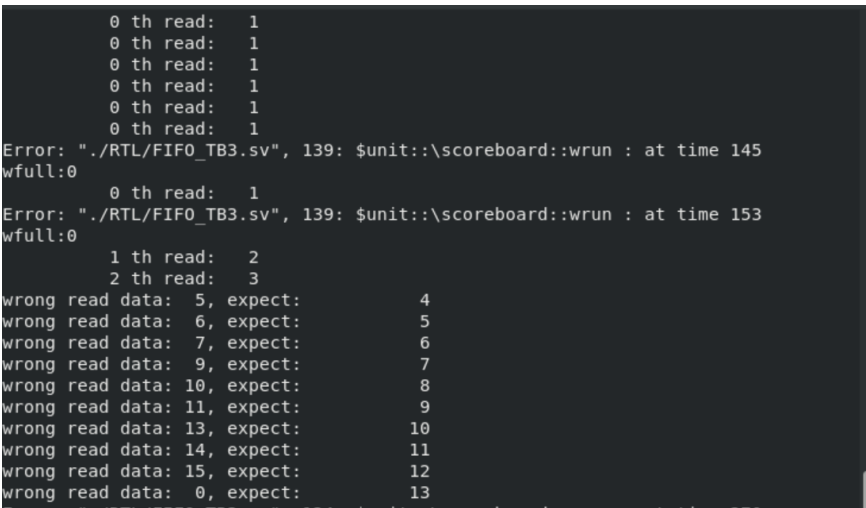


Figure1: terminal output

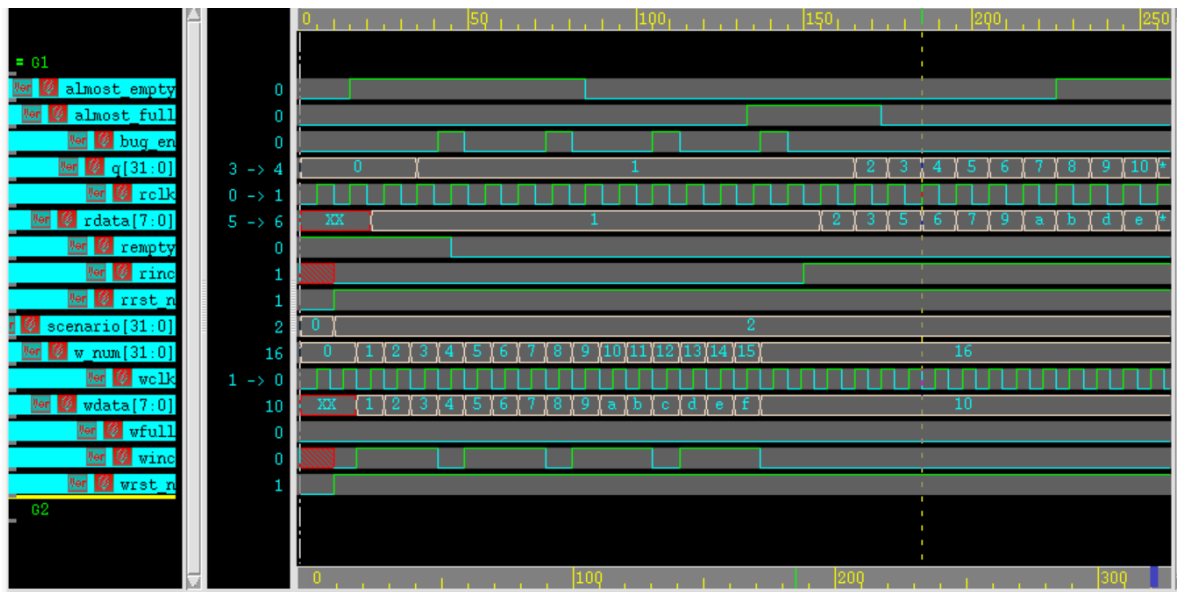


Figure 2: Verdi simulation waveform, data 4, 8, 12 and 16 are dropped, and the fifo does not contain 16 data to be read out, so almost empty flag is asserted early.

Read assertion:

Assertion:

```
interface r_dut_if();
  logic [`DSIZE-1:0] rdata;
  logic rinc,rclk,rrst_n;
  logic almost_empty, rempty;
  logic [`ASIZE-1:0] raddr;
  sequence rAddress;
    $stable(raddr);
  endsequence
  property stop_read;
    @(posedge rclk)
    (rempty&rinc)|->rAddress until $fell(rempty);
  endproperty
  aempty: assert property (stop_read);
endinterface
```

Injected error:

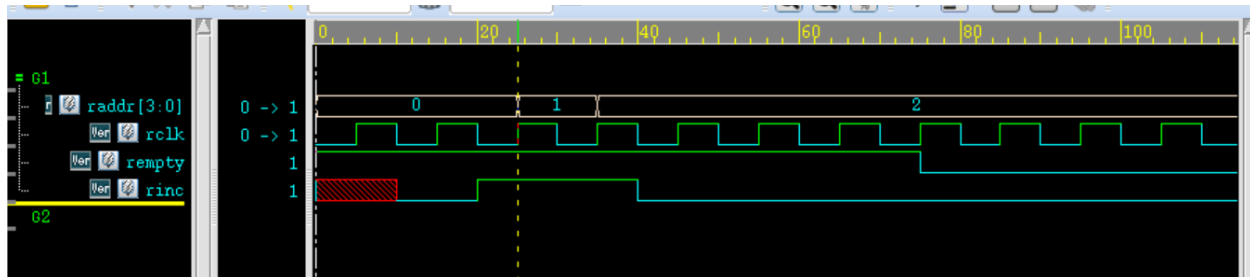
```
    b_wptr[1]=b_wptr[1+1] + q2_wptr[1],
  end
  rbinnext = rbin + (rinc & ~rempty);
  raddr_next = raddr+rinc;
  if (b_wptr-rbinnext<=GAP)begin
```

Result:

```

0 th read: 0
"FIFO_TB3_assertion.sv", 24: FIFO_testbench.r_dut_if_inst.aempty: started at 35s
failed at 35s
"FIFO_TB3_assertion.sv", 24: FIFO_testbench.r_dut_if_inst.aempty: started at 25s
failed at 35s
0 th read: 0

```



Write assertion:

Assertion:

```

interface w_dut_if();
    logic [`DSIZE-1:0] wdata;
    logic winc, wrst_n, wclk;
    logic almost_full, wfull;
    logic bug_en;
    logic [`ASIZE-1:0] waddr;
    sequence wAddress;
        $stable(waddr);
    endsequence
    property stop_write;
        @(posedge wclk)
        (wfull & winc) | => wAddress until $fell(wfull);
    endproperty
    afull: assert property (stop_write);
endinterface

```

Injected error:

```

        b_rptr[i]=b_rptr[i+1]^wq2_rptr[i];
    end
    wbinnext = wbin + (winc & ~wfull);
    waddr_next=waddr+winc;
    if (wbinnext-b_rptr>=GAP)begin
        almost_full_val = 1'b1;
    end

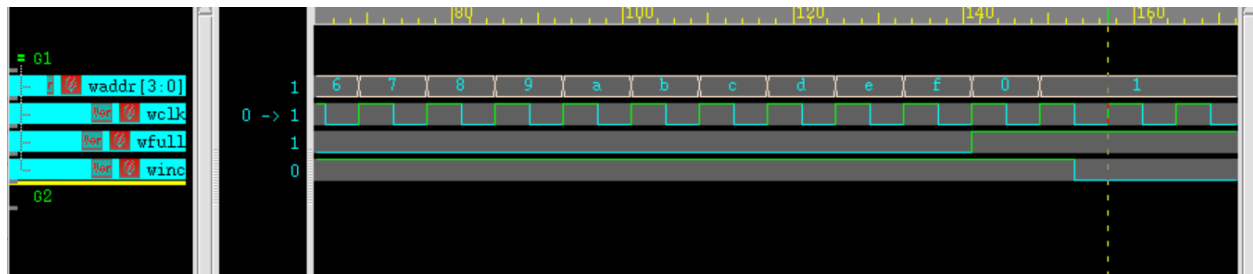
```

Result:

```

0 th read: 1
0 th read: 1
0 th read: 1
"FIFO_TB3_assertion.sv", 40: FIFO_testbench.w_dut_if_inst.afull: started at 149s
failed at 157s
0 th read: 1
1 th read: 2
Error: "FIFO_TB3_assertion.sv", 157: $unit::\scoreboard::wrun : at time 177
wfull:1

```



ADDV Lab #4 demo



Ruthwik
组织者



5 分钟



线上会议
详情待确认后提供



六月 26 日 周四
17:40 – 17:45



Phoenix

已安排会议

我们已向Ruthwik发送了会议确认电子邮件，并已将会议添加到你的日历。